

Chapter 9. Intermediate Memory Topics

How a Computer Views Memory

Let's review how memory within a computer works. You may also want to re-read Chapter 2.

A computer looks at memory as a long sequence of numbered storage locations. A sequence of *millions* of numbered storage locations. Everything is stored in these locations. Your programs are stored there, your data is stored there, everything. Each storage location looks like every other one. The locations holding your program are just like the ones holding your data. In fact, the computer has no idea which are which, except that the executable file tells it where to start executing.

These storage locations are called bytes. The computer can combine up to four of them together into a single word. Normally numeric data is operated on a word at a time. As we mentioned, instructions are also stored in this same memory. Each instruction is a different length. Most instructions take up one or two storage locations for the instruction itself, and then storage locations for the instruction's arguments. For example, the instruction

```
movl data_items(,%edi,4), %ebx
```

takes up 7 storage locations. The first two hold the instruction, the third one tells which registers to use, and the next four hold the storage location of `data_items`. In memory, instructions look just like all the other numbers, and the instructions themselves can be moved into and out of registers just like numbers, because that's what they are.

This chapter is focused on the details of computer memory. To get started let's review some basic terms that we will be using in this chapter:

Chapter 9. Intermediate Memory Topics

Byte

This is the size of a storage location. On x86 processors, a byte can hold numbers between 0 and 255.

Word

This is the size of a normal register. On x86 processors, a word is four bytes long. Most computer operations handle a word at a time.

Address

An address is a number that refers to a byte in memory. For example, the first byte on a computer has an address of 0, the second has an address of 1, and so on.¹ Every piece of data on the computer not in a register has an address. The address of data which spans several bytes is the same as the address of its first byte.

Normally, we don't ever type the numeric address of anything, but we let the assembler do it for us. When we use labels in code, the symbol used in the label will be equivalent to the address it is labelling. The assembler will then replace that symbol with its address wherever you use it in your program. For example, say you have the following code:

```
.section .data
my_data:
    .long 2, 3, 4
```

Now, any time in the program that `my_data` is used, it will be replaced by the address of the first value of the `.long` directive.

Pointer

A pointer is a register or memory word whose value is an address. In our programs we use `%ebp` as a pointer to the current stack frame. All base

1. You actually never use addresses this low, but it works for discussion.

pointer addressing involves pointers. Programming uses a lot of pointers, so it's an important concept to grasp.

The Memory Layout of a Linux Program

When your program is loaded into memory, each `.section` is loaded into its own region of memory. All of the code and data declared in each section is brought together, even if they were separated in your source code.

The actual instructions (the `.text` section) are loaded at the address `0x08048000` (numbers starting with `0x` are in hexadecimal, which will be discussed in Chapter 10). The `.data` section is loaded immediately after that, followed by the `.bss` section.

The last byte that can be addressed on Linux is location `0xbfffffff`. Linux starts the stack here and grows it downward toward the other sections. Between them is a huge gap. The initial layout of the stack is as follows: At the bottom of the stack (the bottom of the stack is the top address of memory - see Chapter 4), there is a word of memory that is zero. After that comes the null-terminated name of the program using ASCII characters. After the program name comes the program's environment variables (these are not important to us in this book). Then come the program's command-line arguments. These are the values that the user typed in on the command line to run this program. When we run `as`, for example, we give it several arguments - `as`, `sourcefile.s`, `-o`, and `objectfile.o`. After these, we have the number of arguments that were used. When the program begins, this is where the stack pointer, `%esp`, is pointing. Further pushes on the stack move `%esp` down in memory. For example, the instruction

```
pushl %eax
```

is equivalent to

```
movl %eax, (%esp)
```

Chapter 9. Intermediate Memory Topics

```
subl $4, %esp
```

Likewise, the instruction

```
popl %eax
```

is the same as

```
movl (%esp), %eax  
addl $4, %esp
```

Your program's data region starts at the bottom of memory and goes up. The stack starts at the top of memory, and moves downward with each push. This middle part between the stack and your program's data sections is inaccessible memory - you are not allowed to access it until you tell the kernel that you need it.² If you try, you will get an error (the error message is usually "segmentation fault"). The same will happen if you try to access data before the beginning of your program, 0x08048000. The last accessible memory address to your program is called the *system break* (also called the *current break* or just the *break*).

2. The stack can access it as it grows downward, and you can access the stack regions through %esp. However, your program's data section doesn't grow that way. The way to grow that will be explained shortly.

0xbfffffff

Environment Variables	
...	
Arg #2	
Arg #1	
Program name	
# of arguments	%esp
Unmapped Memory	Break
Program Code and Data	

0x08048000

*Memory Layout of a Linux Program at Startup***Every Memory Address is a Lie**

So, why does the computer not allow you to access memory in the break area? To answer this question, we will have to delve into the depths of how your computer really handles memory.

You may have wondered, since every program gets loaded into the same place in

Chapter 9. Intermediate Memory Topics

memory, don't they step on each other, or overwrite each other? It would seem so. However, as a program writer, you only access *virtual memory*.

Physical memory refers to the actual RAM chips inside your computer and what they contain. It's usually between 16 and 512 Megabytes on modern computers. If we talk about a *physical memory address*, we are talking about where exactly on these chips a piece of memory is located. Virtual memory is the way *your program* thinks about memory. Before loading your program, Linux finds an empty physical memory space large enough to fit your program, and then tells the processor to pretend that this memory is actually at the address 0x0804800 to load your program into. Confused yet? Let me explain further.

Each program gets its own sandbox to play in. Every program running on your computer thinks that it was loaded at memory address 0x0804800, and that its stack starts at 0xbfffffff. When Linux loads a program, it finds a section of unused memory, and then tells the processor to use that section of memory as the address 0x0804800 for this program. The address that a program believes it uses is called the virtual address, while the actual address on the chips that it refers to is called the physical address. The process of assigning virtual addresses to physical addresses is called *mapping*.

Earlier we talked about the inaccessible memory between the .bss and the stack, but we didn't talk about why it was there. The reason is that this region of virtual memory addresses hasn't been mapped onto physical memory addresses. The mapping process takes up considerable time and space, so if every possible virtual address of every possible program were mapped, you would not have enough physical memory to even run one program. So, the break is the beginning of the area that contains unmapped memory. With the stack, however, Linux will automatically map in memory that is accessed from stack pushes.

Of course, this is a very simplified view of virtual memory. The full concept is much more advanced. For example, Virtual memory can be mapped to more than just physical memory; it can be mapped to disk as well. Swap partitions on Linux allow Linux's virtual memory system to map memory not only to physical RAM,

but also to disk blocks as well. For example, let's say you only have 16 Megabytes of physical memory. Let's also say that 8 Megabytes are being used by Linux and some basic applications, and you want to run a program that requires 20 Megabytes of memory. Can you? The answer is yes, but only if you have set up a swap partition. What happens is that after all of your remaining 8 Megabytes of physical memory have been mapped into virtual memory, Linux starts mapping parts of your application's virtual memory to disk blocks. So, if you access a "memory" location in your program, that location may not actually be in memory at all, but on disk. As the programmer you won't know the difference, though, because it is all handled behind the scenes by Linux.

Now, x86 processors cannot run instructions directly from disk, nor can they access data directly from disk. This requires the help of the operating system. When you try to access memory that is mapped to disk, the processor notices that it can't service your memory request directly. It then asks Linux to step in. Linux notices that the memory is actually on disk. Therefore, it moves some data that is currently in memory onto disk to make room, and then moves the memory being accessed from the disk back into physical memory. It then adjusts the processor's virtual-to-physical memory lookup tables so that it can find the memory in the new location. Finally, Linux returns control to the program and restarts it at the instruction which was trying to access the data in the first place. This instruction can now be completed successfully, because the memory is now in physical RAM.³

Here is an overview of the way memory accesses are handled under Linux:

- The program tries to load memory from a virtual address.
- The processor, using tables supplied by Linux, transforms the virtual memory address into a physical memory address on the fly.

3. Note that not only can Linux have a virtual address map to a different physical address, it can also move those mappings around as needed.

Chapter 9. Intermediate Memory Topics

- If the processor does not have a physical address listed for the memory address, it sends a request to Linux to load it.
- Linux looks at the address. If it is mapped to a disk location, it continues on to the next step. Otherwise, it terminates the program with a segmentation fault error.
- If there is not enough room to load the memory from disk, Linux will move another part of the program or another program onto disk to make room.
- Linux then moves the data into a free physical memory address.
- Linux updates the processor's virtual-to-physical memory mapping tables to reflect the changes.
- Linux restores control to the program, causing it to re-issue the instruction which caused this process to happen.
- The processor can now handle the instruction using the newly-loaded memory and translation tables.

It's a lot of work for the operating system, but it gives the user and the programmer great flexibility when it comes to memory management.

Now, in order to make the process more efficient, memory is separated out into groups called *pages*. When running Linux on x86 processors, a page is 4096 bytes of memory. All of the memory mappings are done a page at a time. Physical memory assignment, swapping, mapping, etc. are all done to memory pages instead of individual memory addresses. What this means to you as a programmer is that whenever you are programming, you should try to keep most memory accesses within the same basic range of memory, so you will only need a page or two of memory at a time. Otherwise, Linux may have to keep moving pages on and off of disk to satisfy your memory needs. Disk access is slow, so this can really slow down your program.

Sometimes so many programs can be loaded that there is hardly enough physical memory for them. They wind up spending more time just swapping memory on

and off of disk than they do actually processing it. This leads to a condition called *swap death* which leads to your system being unresponsive and unproductive. It's usually usually recoverable if you start terminating your memory-hungry programs, but it's a pain.

Resident Set Size: The amount of memory that your program currently has in physical memory is called it's resident set size, and can be viewed by using the program `top`. The resident set size is listed under the column labelled "RSS".

Getting More Memory

We now know that Linux maps all of our virtual memory into physical memory or swap. If you try to access a piece of virtual memory that hasn't been mapped yet, it triggers an error known as a segmentation fault, which will terminate your program. The program break point, if you remember, is the last valid address you can use. Now, this is all great if you know beforehand how much storage you will need. You can just add all the memory you need to your `.data` or `.bss` sections, and it will all be there. However, let's say you don't know how much memory you will need. For example, with a text editor, you don't know how long the person's file will be. You could try to find a maximum file size, and just tell the user that they can't go beyond that, but that's a waste if the file is small. Therefore Linux has a facility to move the break point to accomodate an application's memory needs.

If you need more memory, you can just tell Linux where you want the new break point to be, and Linux will map all the memory you need between the current and new break point, and then move the break point to the spot you specify. That memory is now available for your program to use. The way we tell Linux to move the break point is through the `brk` system call. The `brk` system call is call number

Chapter 9. Intermediate Memory Topics

45 (which will be in `%eax`). `%ebx` should be loaded with the requested breakpoint. Then you call `int $0x80` to signal Linux to do its work. After mapping in your memory, Linux will return the new break point in `%eax`. The new break point might actually be larger than what you asked for, because Linux rounds up to the nearest page. If there is not enough physical memory or swap to fulfill your request, Linux will return a zero in `%eax`. Also, if you call `brk` with a zero in `%ebx`, it will simply return the last usable memory address.

The problem with this method is keeping track of the memory we request. Let's say I need to move the break to have room to load a file, and then need to move a break again to load another file. Let's say I then get rid of the first file. You now have a giant gap in memory that's mapped, but that you aren't using. If you continue to move the break in this way for each file you load, you can easily run out of memory. So, what is needed is a *memory manager*.

A memory manager is a set of routines that takes care of the dirty work of getting your program memory for you. Most memory managers have two basic functions - `allocate` and `deallocate`.⁴ Whenever you need a certain amount of memory, you can simply tell `allocate` how much you need, and it will give you back an address to the memory. When you're done with it, you tell `deallocate` that you are through with it. `allocate` will then be able to reuse the memory. This pattern of memory management is called *dynamic memory allocation*. This minimizes the number of "holes" in your memory, making sure that you are making the best use of it you can. The pool of memory used by memory managers is commonly referred to as *the heap*.

The way memory managers work is that they keep track of where the system break is, and where the memory that you have allocated is. They mark each block of memory in the heap as being used or unused. When you request memory, the memory manager checks to see if there are any unused blocks of the appropriate size. If not, it calls the `brk` system call to request more memory. When you free

4. The function names usually aren't `allocate` and `deallocate`, but the functionality will be the same. In the C programming language, for example, they are named `malloc` and `free`.

memory it marks the block as unused so that future requests can retrieve it. In the next section we will look at building our own memory manager.

A Simple Memory Manager

Here I will show you a simple memory manager. It is very primitive but it shows the principles quite well. As usual, I will give you the program first for you to look through. Afterwards will follow an in-depth explanation. It looks long, but it is mostly comments.

```
#PURPOSE: Program to manage memory usage - allocates
#         and deallocates memory as requested
#
#NOTES:   The programs using these routines will ask
#         for a certain size of memory. We actually
#         use more than that size, but we put it
#         at the beginning, before the pointer
#         we hand back. We add a size field and
#         an AVAILABLE/UNAVAILABLE marker. So, the
#         memory looks like this
#
# #####
# #Available Marker#Size of memory#Actual memory locations#
# #####
#                                     ^--Returned pointer
#                                     points here
#
#     The pointer we return only points to the actual
#     locations requested to make it easier for the
#     calling program. It also allows us to change our
#     structure without the calling program having to
#     change at all.

.section .data
```

Chapter 9. Intermediate Memory Topics

#####GLOBAL VARIABLES#####

#This points to the beginning of the memory we are managing
heap_begin:

.long 0

#This points to one location past the memory we are managing
current_break:

.long 0

#####STRUCTURE INFORMATION####

#size of space for memory region header

.equ HEADER_SIZE, 8

#Location of the "available" flag in the header

.equ HDR_AVAIL_OFFSET, 0

#Location of the size field in the header

.equ HDR_SIZE_OFFSET, 4

#####CONSTANTS#####

.equ UNAVAILABLE, 0 #This is the number we will use to mark
#space that has been given out

.equ AVAILABLE, 1 #This is the number we will use to mark
#space that has been returned, and is
#available for giving

.equ SYS_BRK, 45 #system call number for the break
#system call

.equ LINUX_SYSCALL, 0x80 #make system calls easier to read

.section .text

```
#####FUNCTIONS#####
```

```
##allocate_init##
#PURPOSE: call this function to initialize the
#         functions (specifically, this sets heap_begin and
#         current_break). This has no parameters and no
#         return value.
.globl allocate_init
.type allocate_init,@function
allocate_init:
    pushl %ebp                #standard function stuff
    movl  %esp, %ebp

    #If the brk system call is called with 0 in %ebx, it
    #returns the last valid usable address
    movl  $SYS_BRK, %eax      #find out where the break is
    movl  $0, %ebx
    int   $LINUX_SYSCALL

    incl  %eax                #%eax now has the last valid
                                #address, and we want the
                                #memory location after that

    movl  %eax, current_break #store the current break

    movl  %eax, heap_begin    #store the current break as our
                                #first address. This will cause
                                #the allocate function to get
                                #more memory from Linux the
                                #first time it is run

    movl  %ebp, %esp          #exit the function
    popl  %ebp
```

Chapter 9. Intermediate Memory Topics

```
ret
#####END OF FUNCTION#####

##allocate##
#PURPOSE:      This function is used to grab a section of
#               memory. It checks to see if there are any
#               free blocks, and, if not, it asks Linux
#               for a new one.
#
#PARAMETERS:   This function has one parameter - the size
#               of the memory block we want to allocate
#
#RETURN VALUE:
#               This function returns the address of the
#               allocated memory in %eax. If there is no
#               memory available, it will return 0 in %eax
#
#####PROCESSING#####
#Variables used:
#
#   %ecx - hold the size of the requested memory
#           (first/only parameter)
#   %eax - current memory region being examined
#   %ebx - current break position
#   %edx - size of current memory region
#
#We scan through each memory region starting with
#heap_begin. We look at the size of each one, and if
#it has been allocated. If it's big enough for the
#requested size, and its available, it grabs that one.
#If it does not find a region large enough, it asks
#Linux for more memory. In that case, it moves
#current_break up
```

```

.globl allocate
.type allocate,@function
.equ ST_MEM_SIZE, 8 #stack position of the memory size
                      #to allocate

allocate:
    pushl %ebp                #standard function stuff
    movl  %esp, %ebp

    movl  ST_MEM_SIZE(%ebp), %ecx #%ecx will hold the size
                                #we are looking for (which is the first
                                #and only parameter)

    movl  heap_begin, %eax     #%eax will hold the current
                                #search location

    movl  current_break, %ebx  #%ebx will hold the current
                                #break

alloc_loop_begin:             #here we iterate through each
                                #memory region

    cmpl  %ebx, %eax          #need more memory if these are equal
    je    move_break

    #grab the size of this memory
    movl  HDR_SIZE_OFFSET(%eax), %edx
    #If the space is unavailable, go to the
    cmpl  $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
    je    next_location      #next one

    cmpl  %edx, %ecx          #If the space is available, compare
    jle   allocate_here      #the size to the needed size. If its
                                #big enough, go to allocate_here

```

Chapter 9. Intermediate Memory Topics

```
next_location:
    addl  $HEADER_SIZE, %eax #The total size of the memory
    addl  %edx, %eax         #region is the sum of the size
                             #requested (currently stored
                             #in %edx), plus another 8 bytes
                             #for the header (4 for the
                             #AVAILABLE/UNAVAILABLE flag,
                             #and 4 for the size of the
                             #region). So, adding %edx and $8
                             #to %eax will get the address
                             #of the next memory region

    jmp   alloc_loop_begin   #go look at the next location

allocate_here:               #if we've made it here,
                             #that means that the
                             #region header of the region
                             #to allocate is in %eax

    #mark space as unavailable
    movl  $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
    addl  $HEADER_SIZE, %eax  #move %eax past the header to
                             #the usable memory (since
                             #that's what we return)

    movl  %ebp, %esp         #return from the function
    popl  %ebp
    ret

move_break:                 #if we've made it here, that
                             #means that we have exhausted
                             #all addressable memory, and
                             #we need to ask for more.
                             #%ebx holds the current
```


Chapter 9. Intermediate Memory Topics

```
#endpoint of the data,
#and %ecx holds its size

#we need to increase %ebx to
#where we _want_ memory
#to end, so we
addl $HEADER_SIZE, %ebx #add space for the headers
                          #structure
addl %ecx, %ebx          #add space to the break for
                          #the data requested

#now its time to ask Linux
#for more memory

pushl %eax               #save needed registers
pushl %ecx
pushl %ebx

movl $SYS_BRK, %eax      #reset the break (%ebx has
                          #the requested break point)
int $LINUX_SYSCALL
                          #under normal conditions, this should
                          #return the new break in %eax, which
                          #will be either 0 if it fails, or
                          #it will be equal to or larger than
                          #we asked for. We don't care
                          #in this program where it actually
                          #sets the break, so as long as %eax
                          #isn't 0, we don't care what it is

cmpl $0, %eax            #check for error conditions
je error

popl %ebx                #restore saved registers
popl %ecx
```

Chapter 9. Intermediate Memory Topics

```
popl    %eax

#set this memory as unavailable, since we're about to
#give it away
movl    $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
#set the size of the memory
movl    %ecx, HDR_SIZE_OFFSET(%eax)

#move %eax to the actual start of usable memory.
#%eax now holds the return value
addl    $HEADER_SIZE, %eax

movl    %ebx, current_break    #save the new break

movl    %ebp, %esp             #return the function
popl    %ebp
ret

error:
movl    $0, %eax               #on error, we return zero
movl    %ebp, %esp
popl    %ebp
ret
#####END OF FUNCTION#####

##deallocate##
#PURPOSE:
#    The purpose of this function is to give back
#    a region of memory to the pool after we're done
#    using it.
#
#PARAMETERS:
#    The only parameter is the address of the memory
#    we want to return to the memory pool.
```

```

#
#RETURN VALUE:
#   There is no return value
#
#PROCESSING:
#   If you remember, we actually hand the program the
#   start of the memory that they can use, which is
#   8 storage locations after the actual start of the
#   memory region. All we have to do is go back
#   8 locations and mark that memory as available,
#   so that the allocate function knows it can use it.
.globl deallocate
.type deallocate,@function
#stack position of the memory region to free
.equ ST_MEMORY_SEG, 4
deallocate:
#since the function is so simple, we
#don't need any of the fancy function stuff

#get the address of the memory to free
#(normally this is 8(%ebp), but since
#we didn't push %ebp or move %esp to
#%ebp, we can just do 4(%esp)
movl ST_MEMORY_SEG(%esp), %eax

#get the pointer to the real beginning of the memory
subl $HEADER_SIZE, %eax

#mark it as available
movl $AVAILABLE, HDR_AVAIL_OFFSET(%eax)

#return
ret
#####END OF FUNCTION#####

```

The first thing to notice is that there is no `_start` symbol. The reason is that this is just a set of functions. A memory manager by itself is not a full program - it doesn't do anything. It is simply a utility to be used by other programs.

To assemble the program, do the following:

```
as alloc.s -o alloc.o
```

Okay, now let's look at the code.

Variables and Constants

At the beginning of the program, we have two locations set up:

```
heap_begin:
    .long 0
```

```
current_break:
    .long 0
```

Remember, the section of memory being managed is commonly referred to as the *heap*. When we assemble the program, we have no idea where the beginning of the heap is, nor where the current break is. Therefore, we reserve space for their addresses, but just fill them with a 0 for the time being.

Next we have a set of constants to define the structure of the heap. The way this memory manager works is that before each region of memory allocated, we will have a short record describing the memory. This record has a word reserved for the available flag and a word for the region's size. The actual memory allocated immediately follows this record. The available flag is used to mark whether this region is available for allocations, or if it is currently in use. The size field lets us know both whether or not this region is big enough for an allocation request, as

well as the location of the next memory region. The following constants describe this record:

```
.equ HEADER_SIZE, 8
.equ HDR_AVAIL_OFFSET, 0
.equ HDR_SIZE_OFFSET, 4
```

This says that the header is 8 bytes total, the available flag is offset 0 bytes from the beginning, and the size field is offset 4 bytes from the beginning. If we are careful to always use these constants, then we protect ourselves from having to do too much work if we later decide to add more information to the header.

The values that we will use for our `available` field are either 0 for unavailable, or 1 for available. To make this easier to read, we have the following definitions:

```
.equ UNAVAILABLE, 0
.equ AVAILABLE, 1
```

Finally, we have our Linux system call definitions:

```
.equ BRK, 45
.equ LINUX_SYSCALL, 0x80
```

The `allocate_init` function

Okay, this is a simple function. All it does is set up the `heap_begin` and `current_break` variables we discussed earlier. So, if you remember the discussion earlier, the current break can be found using the `brk` system call. So, the function starts like this:

```
pushl %ebp
movl %esp, %ebp

movl $SYS_BRK, %eax
```

Chapter 9. Intermediate Memory Topics

```
movl    $0,    %ebx
int      $LINUX_SYSCALL
```

Anyway, after `int $LINUX_SYSCALL`, `%eax` holds the last valid address. We actually want the first invalid address instead of the last valid address, so we just increment `%eax`. Then we move that value to the `heap_begin` and `current_break` locations. Then we leave the function. The code looks like this:

```
incl    %eax
movl    %eax,  current_break
movl    %eax,  heap_begin
movl    %ebp,  %esp
popl    %ebp
ret
```

The heap consists of the memory between `heap_begin` and `current_break`, so this says that we start off with a heap of zero bytes. Our `allocate` function will then extend the heap as much as it needs to when it is called.

The `allocate` function

This is the doozy function. Let's start by looking at an outline of the function:

1. Start at the beginning of the heap.
2. Check to see if we're at the end of the heap.
3. If we are at the end of the heap, grab the memory we need from Linux, mark it as "unavailable" and return it. If Linux won't give us any more, return a 0.
4. If the current memory region is marked "unavailable", go to the next one, and go back to step 2.
5. If the current memory region is too small to hold the requested amount of space, go back to step 2.

6. If the memory region is available and large enough, mark it as "unavailable" and return it.

Now, look back through the code with this in mind. Be sure to read the comments so you'll know which register holds which value.

Now that you've looked back through the code, let's examine it one line at a time. We start off like this:

```
pushl %ebp
movl  %esp, %ebp
movl  ST_MEM_SIZE(%ebp), %ecx
movl  heap_begin, %eax
movl  current_break, %ebx
```

This part initializes all of our registers. The first two lines are standard function stuff. The next move pulls the size of the memory to allocate off of the stack. This is our only function parameter. After that, it moves the beginning heap address and the end of the heap into registers. I am now ready to do processing.

The next section is marked `alloc_loop_begin`. In this loop we are going to examine memory regions until we either find an open memory region or determine that we need more memory. Our first instructions check to see if we need more memory:

```
cmpl %ebx, %eax
je   move_break
```

`%eax` holds the current memory region being examined and `%ebx` holds the location past the end of the heap. Therefore if the next region to be examined is past the end of the heap, it means we need more memory to allocate a region of this size. Let's skip down to `move_break` and see what happens there:

```
move_break:
addl  $HEADER_SIZE, %ebx
addl  %ecx, %ebx
```

Chapter 9. Intermediate Memory Topics

```
pushl %eax
pushl %ecx
pushl %ebx
movl  $SYS_BRK, %eax
int   $LINUX_SYSCALL
```

When we reach this point in the code, %ebx holds where we want the next region of memory to be. So, we add our header size and region size to %ebx, and that's where we want the system break to be. We then push all the registers we want to save on the stack, and call the brk system call. After that we check for errors:

```
cmpl  $0, %eax
je    error
```

If there were no errors we pop the registers back off the stack, mark the memory as unavailable, record the size of the memory, and make sure %eax points to the start of usable memory (which is *after* the header).

```
popl  %ebx
popl  %ecx
popl  %eax
movl  $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
movl  %ecx, HDR_SIZE_OFFSET(%eax)
addl  $HEADER_SIZE, %eax
```

Then we store the new program break and return the pointer to the allocated memory.

```
movl  %ebx, current_break
movl  %ebp, %esp
popl  %ebp
ret
```

The error code just returns 0 in %eax, so we won't discuss it.

Let's go back look at the rest of the loop. What happens if the current memory being looked at isn't past the end of the heap? Well, let's look.

```
movl HDR_SIZE_OFFSET(%eax), %edx
cmpl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
je   next_location
```

This first grabs the size of the memory region and puts it in %edx. Then it looks at the available flag to see if it is set to UNAVAILABLE. If so, that means that memory region is in use, so we'll have to skip over it. So, if the available flag is set to UNAVAILABLE, you go to the code labeled next_location. If the available flag is set to AVAILABLE, then we keep on going.

Let's say that the space was available, and so we keep going. Then we check to see if this space is big enough to hold the requested amount of memory. The size of this region is being held in %edx, so we do this:

```
cmpl %edx, %ecx
jle  allocate_here
```

If the requested size is less than or equal to the current region's size, we can use this block. It doesn't matter if the current region is larger than requested, because the extra space will just be unused. So, let's jump down to allocate_here and see what happens:

```
movl $UNAVAILABLE, HDR_AVAIL_OFFSET(%eax)
addl $HEADER_SIZE, %eax
movl %ebp, %esp
popl %ebp
ret
```

It marks the memory as being unavailable. Then it moves the pointer %eax past the header, and uses it as the return value for the function. Remember, the person using this function doesn't need to even know about our memory header record. They just need a pointer to usable memory.

Chapter 9. Intermediate Memory Topics

Okay, so let's say the region wasn't big enough. What then? Well, we would then be at the code labeled `next_location`. This section of code is used any time that we figure out that the current memory region won't work for allocating memory. All it does is advance `%eax` to the next possible memory region, and goes back to the beginning of the loop. Remember that `%edx` is holding the size of the current memory region, and `HEADER_SIZE` is the symbol for the size of the memory region's header. So this code will move us to the next memory region:

```
addl $HEADER_SIZE, %eax
addl %edx, %eax
jmp  alloc_loop_begin
```

And now the function runs another loop.

Whenever you have a loop, you must make sure that it will *always* end. The best way to do that is to examine all of the possibilities, and make sure that all of them eventually lead to the loop ending. In our case, we have the following possibilities:

- We will reach the end of the heap
- We will find a memory region that's available and large enough
- We will go to the next location

The first two items are conditions that will cause the loop to end. The third one will keep it going. However, even if we never find an open region, we will eventually reach the end of the heap, because it is a finite size. Therefore, we know that no matter which condition is true, the loop has to eventually hit a terminating condition.

The `deallocate` function

The `deallocate` function is much easier than the `allocate` one. That's because it doesn't have to do any searching at all. It can just mark the current memory

region as AVAILABLE, and `allocate` will find it next time it is called. So we have:

```
movl  ST_MEMORY_SEG(%esp), %eax
subl  $HEADER_SIZE, %eax
movl  $AVAILABLE, HDR_AVAIL_OFFSET(%eax)
ret
```

In this function, we don't have to save `%ebp` or `%esp` since we're not changing them, nor do we have to restore them at the end. All we're doing is reading the address of the memory region from the stack, backing up to the beginning of the header, and marking the region as available. This function has no return value, so we don't care what we leave in `%eax`.

Performance Issues and Other Problems

Our simplistic memory manager is not really useful for anything more than an academic exercise. This section looks at the problems with such a simplistic allocator.

The biggest problem here is speed. Now, if there are only a few allocations made, then speed won't be a big issue. But think about what happens if you make a thousand allocations. On allocation number 1000, you have to search through 999 memory regions to find that you have to request more memory. As you can see, that's getting pretty slow. In addition, remember that Linux can keep pages of memory on disk instead of in memory. So, since you have to go through every piece of memory your program's memory, that means that Linux has to load every part of memory that's currently on disk to check to see if its available. You can see how this could get really, really slow.⁵ This method is said to run in *linear* time, which means that every element you have to manage makes your program take

5. This is why adding more memory to your computer makes it run faster. The more memory your computer has, the less it puts on disk, so it doesn't have to always be interrupting your programs to retrieve pages off the disk.

Chapter 9. Intermediate Memory Topics

longer. A program that runs in *constant* time takes the same amount of time no matter how many elements you are managing. Take the `deallocate` function, for instance. It only runs 4 instructions, no matter how many elements we are managing, or where they are in memory. In fact, although our `allocate` function is one of the slowest of all memory managers, the `deallocate` function is one of the fastest.

Another performance problem is the number of times we're calling the `brk` system call. System calls take a long time. They aren't like functions, because the processor has to switch modes. Your program isn't allowed to map itself memory, but the Linux kernel is. So, the processor has to switch into *kernel mode*, then Linux maps the memory, and then switches back to *user mode* for your application to continue running. This is also called a *context switch*. Context switches are relatively slow on x86 processors. Generally, you should avoid calling the kernel unless you really need to.

Another problem that we have is that we aren't recording where Linux actually sets the break. Previously we mentioned that Linux might actually set the break past where we requested it. In this program, we don't even look at where Linux actually sets the break - we just assume it sets it where we requested. That's not really a bug, but it will lead to unnecessary `brk` system calls when we already have the memory mapped in.

Another problem we have is that if we are looking for a 5-byte region of memory, and the first open one we come to is 1000 bytes, we will simply mark the whole thing as allocated and return it. This leaves 995 bytes of unused, but allocated, memory. It would be nice in such situations to break it apart so the other 995 bytes can be used later. It would also be nice to combine consecutive free spaces when looking for large allocations.

Using our Allocator

The programs we do in this book aren't complicated enough to necessitate a memory manager. Therefore, we will just use our memory manager to allocate a buffer for one of our file reading/writing programs instead of assigning it in the `.bss`.

The program we will demonstrate this on is `read-records.s` from Chapter 6. This program uses a buffer named `record_buffer` to handle its input/output needs. We will simply change this from being a buffer defined in `.bss` to being a pointer to a dynamically-allocated buffer using our memory manager. You will need to have the code from that program handy as we will only be discussing the changes in this section.

The first change we need to make is in the declaration. Currently it looks like this:

```
.section .bss
.lcomm, record_buffer, RECORD_SIZE
```

It would be a misnomer to keep the same name, since we are switching it from being an actual buffer to being a pointer to a buffer. In addition, it now only needs to be one word big (enough to hold a pointer). The new declaration will stay in the `.data` section and look like this:

```
record_buffer_ptr:
.long 0
```

Our next change is we need to initialize our memory manager immediately after we start our program. Therefore, right after the stack is set up, the following call needs to be added:

```
call allocate_init
```

After that, the memory manager is ready to start servicing memory allocation requests. We need to allocate enough memory to hold these records that we are

Chapter 9. Intermediate Memory Topics

reading. Therefore, we will call `allocate` to allocate this memory, and then save the pointer it returns into `record_buffer_ptr`. Like this:

```
pushl $RECORD_SIZE
call  allocate
movl  %eax, record_buffer_ptr
```

Now, when we make the call to `read_record`, it is expecting a pointer. In the old code, the pointer was the immediate-mode reference to `record_buffer`. Now, `record_buffer_ptr` just holds the pointer rather than the buffer itself. Therefore, we must do a direct mode load to get the value in `record_buffer_ptr`. We need to remove this line:

```
pushl $record_buffer
```

And put this line in its place:

```
pushl record_buffer_ptr
```

The next change comes when we are trying to find the address of the `firstname` field of our record. In the old code, it was `$RECORD_FIRSTNAME + record_buffer`. However, that only works because it is a constant offset from a constant address. In the new code, it is the offset of an address stored in `record_buffer_ptr`. To get that value, we will need to move the pointer into a register, and then add `$RECORD_FIRSTNAME` to it to get the pointer. So where we have the following code:

```
pushl $RECORD_FIRSTNAME + record_buffer
```

We need to replace it with this:

```
movl  record_buffer_ptr, %eax
addl  $RECORD_FIRSTNAME, %eax
pushl %eax
```

Similarly, we need to change the line that says

```
movl $RECORD_FIRSTNAME + record_buffer, %ecx
```

so that it reads like this:

```
movl record_buffer_ptr, %ecx  
addl $RECORD_FIRSTNAME, %ecx
```

Finally, one change that we need to make is to deallocate the memory once we are done with it (in this program it's not necessary, but it's a good practice anyway). To do that, we just send `record_buffer_ptr` to the `dealloc` function right before exiting:

```
pushl record_buffer_ptr  
call dealloc
```

Now you can build your program with the following commands:

```
as read-records.s -o read-records.o  
ld alloc.o read-record.o read-records.o write-newline.o count-  
chars.o -o read-records
```

You can then run your program by doing `./read-records`.

The uses of dynamic memory allocation may not be apparent to you at this point, but as you go from academic exercises to real-life programs you will use it continually.

More Information

More information on memory handling in Linux and other operating systems can be found at the following locations:

Chapter 9. Intermediate Memory Topics

- More information about the memory layout of Linux programs can be found in Konstantin Boldyshev's document, "Startup state of a Linux/i386 ELF binary", available at <http://linuxassembly.org/startup.html>
- A good overview of virtual memory in many different systems is available at <http://cne.gmu.edu/modules/vm/>
- Several in-depth articles on Linux's virtual memory subsystem is available at <http://www.nongnu.org/lkdp/files.html>
- Doug Lea has written up a description of his popular memory allocator at <http://gee.cs.oswego.edu/dl/html/malloc.html>
- A paper on the 4.4 BSD memory allocator is available at <http://docs.freebsd.org/44doc/papers/malloc.html>

Review

Know the Concepts

- Describe the layout of memory when a Linux program starts.
- What is the heap?
- What is the current break?
- Which direction does the stack grow in?
- Which direction does the heap grow in?
- What happens when you access unmapped memory?
- How does the operating system prevent processes from writing over each other's memory?
- Describe the process that occurs if a piece of memory you are using is currently residing on disk?

- Why do you need an allocator?

Use the Concepts

- Modify the memory manager so that it calls `allocate_init` automatically if it hasn't been initialized.
- Modify the memory manager so that if the requested size of memory is smaller than the region chosen, it will break up the region into multiple parts. Be sure to take into account the size of the new header record when you do this.
- Modify one of your programs that uses buffers to use the memory manager to get buffer memory rather than using the `.bss`.

Going Further

- Research *garbage collection*. What advantages and disadvantages does this have over the style of memory management used here?
- Research *reference counting*. What advantages and disadvantages does this have over the style of memory management used here?
- Change the name of the functions to `malloc` and `free`, and build them into a shared library. Use `LD_PRELOAD` to force them to be used as your memory manager instead of the default one. Add some `write` system calls to `STDOUT` to verify that your memory manager is being used instead of the default one.

